Титульный лист

Лабораторная работа № 3. Управляющие структуры

Студент: Жаворонков Кирилл Александрович

Группа: НПИбд-01-23

Номер студенческого билета: 1132231844

Дисциплина: Компьютерный практикум на языке Julia

Формулировка задания

Цель работы. Освоить применение циклов и функций, сторонних пакетов Julia для решения задач линейной алгебры и работы с матрицами.

Задание.

1. Используя JupyterLab, повторить примеры из раздела 3.2 методички.

2. Выполнить задания для самостоятельной работы (раздел 3.4):

   • 3.4.1 — вывод чисел от 1 до 100 и их квадратов, словарь и массив квадратов (циклы while и for);

   • 3.4.2 — условный оператор и тернарный оператор для чётных и нечётных чисел;

   • 3.4.3 — функция add_one;

   • 3.4.4 — матрица, каждый элемент которой больше предыдущего на единицу, через map или broadcast;

   • 3.4.5 — степень матрицы A^3 и замена третьего столбца суммой второго и третьего;

   • 3.4.6 — матрица B и произведение $C = B^TB$;

   • 3.4.7 — матрицы Z, E и шаблоны Z1–Z4;

   • 3.4.8 — собственная функция outer(x, y, operation) и матрицы A1–A5;

   • 3.4.9 — решение системы линейных уравнений с 5 неизвестными;

   • 3.4.10 — случайная матрица M размером 6×10 и её анализ;

   • 3.4.11 — вычисление двух двойных сумм.

Выполнение работы

Примеры раздела 3.2 (задание 1)

Циклы for и двумерный массив

В JupyterLab выполнены примеры циклов for: вывод нечётных чисел с шагом 2 и перебор массива строк.

```
for n in 1:2:10
    println(n)
end

myfriends = ["Ted", "Robyn", "Barney", "Lily", "Marshall"]
for friend in myfriends
    println("Hi $friend, it's great to see you!")
```

```
end
```

Результат: вывод чисел 1, 3, 5, 7, 9 и приветствий для каждого из пяти имён.

Затем построены двумерные массивы, в которых значение каждой ячейки равно сумме индексов строки и столбца.

```
m, n = 5, 5
A = fill(0, (m, n))
for i in 1:m
    for j in 1:n
        A[i, j] = i + j
    end
end
A
```

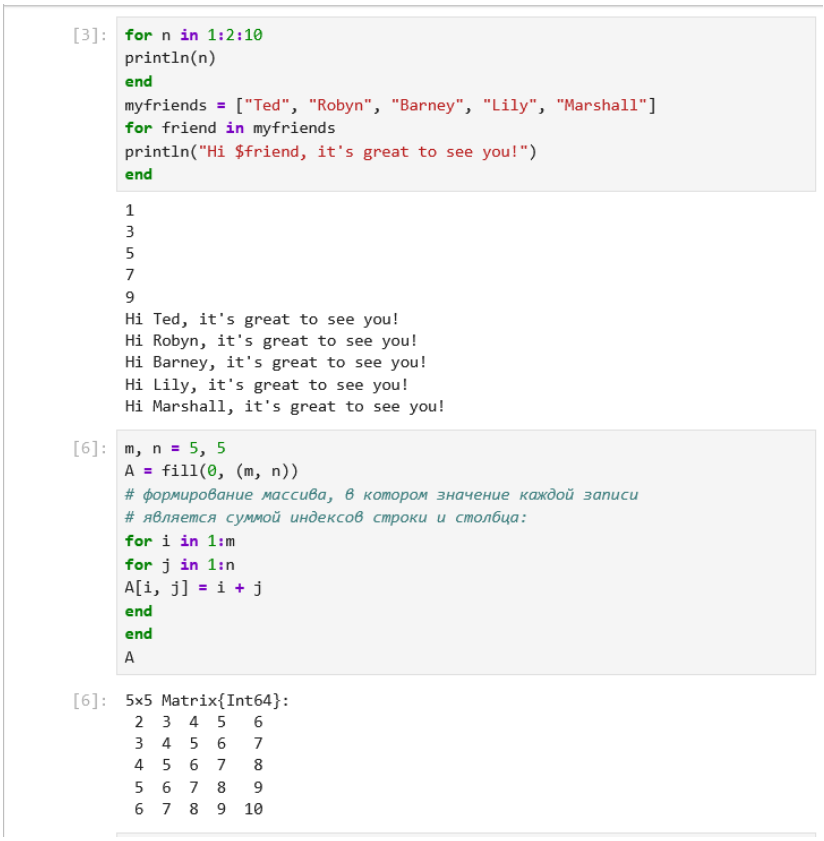


Рисунок 1 — image-12.png

Во втором варианте (B = fill(0, (m, n)), цикл for i in 1:m, j in 1:n) и в генераторе C = [i + j for i in 1:m, j in 1:n] получен тот же массив. Результаты совпадают.

[7]:
```
# инициализация массива m x n из нулей:
B = fill(0, (m, n))
for i in 1:m, j in 1:n
B[i, j] = i + j
end
B
```

[7]:
```
5×5 Matrix{Int64}:
 2  3  4  5   6
 3  4  5  6   7
 4  5  6  7   8
 5  6  7  8   9
 6  7  8  9  10
```

[8]:
```
C = [i + j for i in 1:m, j in 1:n]
C
```

[8]:
```
5×5 Matrix{Int64}:
 2  3  4  5   6
 3  4  5  6   7
 4  5  6  7   8
 5  6  7  8   9
 6  7  8  9  10
```

[15]:
```
# используем `&&` для реализации операции "AND"
# операция % вычисляет остаток от деления
N = 15
if (N % 3 == 0) && (N % 5 == 0)
println("FizzBuzz")
elseif N % 3 == 0
println("Fizz")
elseif N % 5 == 0
println("Buzz")
else
println(N)
end
```

```
FizzBuzz
```

Рисунок 2 — image-11.png

Условные выражения и тернарный оператор

Проверка FizzBuzz для N = 15 дала ответ «FizzBuzz», так как число делится и на 3, и на 5.

```
N = 15
if (N % 3 == 0) && (N % 5 == 0)
    println("FizzBuzz")
elseif N % 3 == 0
    println("Fizz")
elseif N % 5 == 0
    println("Buzz")
else
    println(N)
end
```

Тернарный оператор (x > y) ? x : y при x = 5, y = 10 вернул 10.

```
[16]: x = 5
      y = 10
      (x > y) ? x : y

[16]: 10

[19]: function sayhi(name)
      println("Hi $name, it's great to see you!")
      end

[19]: sayhi (generic function with 1 method)

[20]: function f(x)
      x^2
      end

[20]: f (generic function with 1 method)

[21]: sayhi("C-3PO")
      f(42)

      Hi C-3PO, it's great to see you!
[21]: 1764

[22]: sayhi2(name) = println("Hi $name, it's great to see you!")
      f2(x) = x^2

[22]: f2 (generic function with 1 method)

[23]: sayhi3 = name -> println("Hi $name, it's great to see you!")
      f3 = x -> x^2

[23]: #8 (generic function with 1 method)
```

Рисунок 3 — image-10.png

Функции

Функция sayhi(name) выводит приветствие, функция f(x) = x^2 при x = 42 даёт 1764. Также проверены однострочные и анонимные варианты объявления функций.

```
function sayhi(name)
    println("Hi $name, it's great to see you!")
end

function f(x)
    x^2
end

sayhi("C-3PO")
f(42)
```

Сортировка sort и sort!

Функция sort!(v) изменила массив v на месте, а sort(v) без восклицательного знака вернула новый отсортированный массив и не изменила исходный.

```
v = [3, 5, 2]
sort(v)
v
sort!(v)
v
```

```
[24]: v = [3, 5, 2]
      sort(v)
      v
      sort!(v)
      v
```

```
[24]: 3-element Vector{Int64}:
       2
       3
       5
```

```
[25]: v = [3, 5, 2]
      sort(v)
      v
```

```
[25]: 3-element Vector{Int64}:
       3
       5
       2
```

```
[26]: map(f, [1, 2, 3])
```

```
[26]: 3-element Vector{Int64}:
       1
       4
       9
```

```
[27]: [f(1), f(2), f(3)]
```

```
[27]: 3-element Vector{Int64}:
       1
       4
       9
```

Рисунок 4 — image-9.png

Функции высшего порядка map и broadcast

Функция map(f, [1, 2, 3]) даёт [1, 4, 9], map(x -> x^3, ...) даёт [1, 8, 27]. broadcast(f, ...) и f.(...) дают тот же результат. Для матрицы A = [i + 3*j for j in 0:2, i in 1:3] получена матрица [1 2 3; 4 5 6; 7 8 9].

```
f(x) = x^2
map(f, [1, 2, 3])
map(x -> x^3, [1, 2, 3])
broadcast(f, [1, 2, 3])
f.([1, 2, 3])
A = [i + 3*j for j in 0:2, i in 1:3]
```

```
9
```

```
[28]: f(x) = x^2
      map(f, [1, 2, 3])
```

```
[28]: 3-element Vector{Int64}:
       1
       4
       9
```

```
[29]: x -> x^3
      map(x -> x^3, [1, 2, 3])
```

```
[29]: 3-element Vector{Int64}:
        1
        8
       27
```

```
[30]: f(x) = x^2
      broadcast(f, [1, 2, 3])
```

```
[30]: 3-element Vector{Int64}:
       1
       4
       9
```

```
[31]: f.([1, 2, 3])
```

```
[31]: 3-element Vector{Int64}:
       1
       4
       9
```

```
[32]: A = [i + 3*j for j in 0:2, i in 1:3]
```

```
[32]: 3×3 Matrix{Int64}:
       1  2  3
       4  5  6
       7  8  9
```

Рисунок 5 — image-8.png

Пакет Colors

Пакет Colors подключён, создана палитра из 100 цветов distinguishable_colors(100). Функция rand(palette, 3, 3) возвращает матрицу 3×3 случайных цветов.

```
using Colors
palette = distinguishable_colors(100)
rand(palette, 3, 3)
```



Рисунок 6 — image-7.png

Самостоятельные задания (задание 2)

3.4.1. Циклы while и for

[Скриншот не предоставлен. Вставьте результаты выполнения вывода чисел 1–100 с квадратами, словаря squares и массива squares_arr.]

3.4.2. Условный и тернарный операторы

Для чисел 14 и 15 выполнена проверка чётности условным оператором if. Результат: для 14 выведено число 14, для 15 — строка «нечётное». Тот же результат получен тернарным оператором.

```
num = 14
if num % 2 == 0
    println(num)
else
    println("нечётное")
end

num % 2 == 0 ? println(num) : println("нечётное")
```

[46]:
```
num = 14
if num % 2 == 0
    println(num)
else
    println("нечётное")
end
```

```
14
```

[47]:
```
num = 15
if num % 2 == 0
    println(num)
else
    println("нечётное")
end
```

```
нечётное
```

[48]:
```
num % 2 == 0 ? println(num) : println("нечётное")
```

```
нечётное
```

[49]:
```
add_one(x) = x + 1
println("add_one(5) = ", add_one(5))
```

```
add_one(5) = 6
```

[50]:
```
A_task4 = reshape(map(add_one, 0:8), 3, 3)
println("Матрица A (каждый элемент на 1 больше): \n", A_task4)
```

```
Матрица A (каждый элемент на 1 больше):
[1 4 7; 2 5 8; 3 6 9]
```

Рисунок 7 — image-6.png

3.4.3. Функция add_one

```
add_one(x) = x + 1
println("add_one(5) = ", add_one(5))
```

Результат: add_one(5) = 6.

3.4.4. Матрица с помощью map

Матрица получена применением map(add_one, 0:8) и изменением формы в 3×3. Результат: [1 4 7; 2 5 8; 3 6 9].

```
A_task4 = reshape(map(add_one, 0:8), 3, 3)
```

[46]:
```
num = 14
if num % 2 == 0
    println(num)
else
    println("нечётное")
end
```

```
14
```

[47]:
```
num = 15
if num % 2 == 0
    println(num)
else
    println("нечётное")
end
```

```
нечётное
```

[48]:
```
num % 2 == 0 ? println(num) : println("нечётное")
```

```
нечётное
```

[49]:
```
add_one(x) = x + 1
println("add_one(5) = ", add_one(5))
```

```
add_one(5) = 6
```

[50]:
```
A_task4 = reshape(map(add_one, 0:8), 3, 3)
println("Матрица A (каждый элемент на 1 больше): \n", A_task4)
```

```
Матрица A (каждый элемент на 1 больше):
[1 4 7; 2 5 8; 3 6 9]
```

Рисунок 8 — image-6.png

3.4.5. Степень матрицы и замена столбца

Матрица A = [1 1 3; 5 2 6; -2 -1 -3]. Найдена матричная степень A^3, которая оказалась нулевой матрицей. Затем третий столбец заменён суммой второго и третьего.

```
A_5 = [1 1 3; 5 2 6; -2 -1 -3]
A_cubed = A_5^3
A_5[:, 3] = A_5[:, 2] + A_5[:, 3]
println(A_5)
```

Результат: A^3 = [0 0 0; 0 0 0; 0 0 0], после замены столбца матрица [1 1 4; 5 2 8; -2 -1 -4].

```
[51]: A_5 = [1  1  3;
             5  2  6;
            -2 -1 -3]

      A_cubed = A_5^3
      println("A^3 = \n", A_cubed)

      A_5[:, 3] = A_5[:, 2] + A_5[:, 3]
      println(A_5)
```

```
A^3 =
[0 0 0; 0 0 0; 0 0 0]
[1 1 4; 5 2 8; -2 -1 -4]
```

```
[52]: B = fill(10, 15, 3)
      B[:, 2] .= -10
      C_mat = B' * B
      println("Матрица C = B^T * B: \n", C_mat)
```

```
Матрица C = B^T * B:
[1500 -1500 1500; -1500 1500 -1500; 1500 -1500 1500]
```

```
[53]: Z = zeros(Int, 6, 6)
      E = ones(Int, 6, 6)

      Z1 = zeros(Int, 6, 6)
      for i in 1:6, j in 1:6
          if abs(i - j) == 1
              Z1[i, j] = 1
          end
      end
      println("Матрица Z1:\n", Z1)
```

```
Матрица Z1:
[0 1 0 0 0 0; 1 0 1 0 0 0; 0 1 0 1 0 0; 0 0 1 0 1 0; 0 0 0 1 0 1; 0 0 0 0 1
0]
```

Рисунок 9 — image-4.png

3.4.6. Матрица B и C = $B^T B$

```
B = fill(10, 15, 3)
B[:, 2] .= -10
C_mat = B' * B
```

Результат: C = [1500 -1500 1500; -1500 1500 -1500; 1500 -1500 1500].


```
[51]: A_5 = [1  1  3;
             5  2  6;
            -2 -1 -3]

      A_cubed = A_5^3
      println("A^3 = \n", A_cubed)

      A_5[:, 3] = A_5[:, 2] + A_5[:, 3]
      println(A_5)
```

```
A^3 =
[0 0 0; 0 0 0; 0 0 0]
[1 1 4; 5 2 8; -2 -1 -4]
```

```
[52]: B = fill(10, 15, 3)
      B[:, 2] .= -10
      C_mat = B' * B
      println("Матрица C = B^T * B: \n", C_mat)
```

```
Матрица C = B^T * B:
[1500 -1500 1500; -1500 1500 -1500; 1500 -1500 1500]
```

```
[53]: Z = zeros(Int, 6, 6)
      E = ones(Int, 6, 6)

      Z1 = zeros(Int, 6, 6)
      for i in 1:6, j in 1:6
          if abs(i - j) == 1
              Z1[i, j] = 1
          end
      end
      println("Матрица Z1:\n", Z1)
```

```
Матрица Z1:
[0 1 0 0 0 0; 1 0 1 0 0 0; 0 1 0 1 0 0; 0 0 1 0 1 0; 0 0 0 1 0 1; 0 0 0 0 1
0]
```

Рисунок 10 — image-4.png

3.4.7. Матрицы Z, E и Z1

```
Z = zeros(Int, 6, 6)
E = ones(Int, 6, 6)
Z1 = zeros(Int, 6, 6)
for i in 1:6, j in 1:6
    if abs(i - j) == 1
        Z1[i, j] = 1
    end
end
Z1
```

Результат для Z1 совпадает с эталоном из методички: первая строка [0 1 0 0 0 0], вторая [1 0 1 0 0 0].

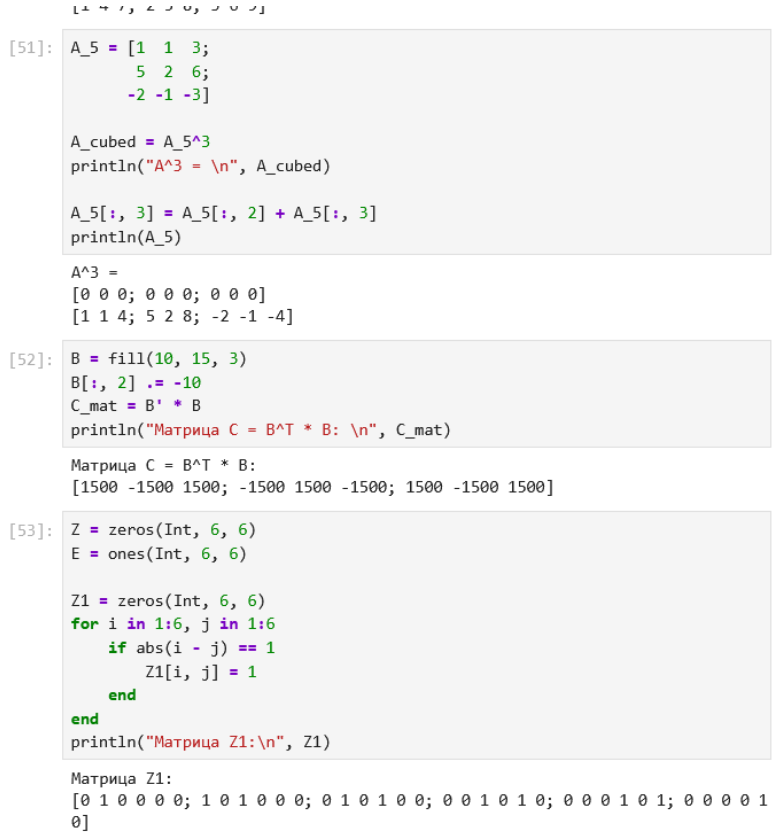


Рисунок 11 — image-4.png

[Скриншоты матриц Z2, Z3, Z4 не предоставлены. Вставьте их после выполнения соответствующих ячеек.]

3.4.8. Функция outer и матрицы A1–A5

Для построения матриц реализована функция outer(x, y, operation), которая применяет operation к каждой паре элементов (i из x, j из y).

```
function outer(x, y, operation)
    return [operation(i, j) for i in x, j in y]
end

A1 = outer(0:4, 0:4, +)
A2 = outer(0:4, 1:5, (x, y) -> x == 0 ? 0 : x^(y-1))
A3 = outer(0:4, 0:4, (x, y) -> (x + y) % 5)
A4 = outer(0:9, 0:9, (x, y) -> (x + y) % 10)
A5 = outer(1:10, 1:10, (x, y) -> (x - y + 10) % 10)
```

Результаты:

• A1 — таблица сумм i + j для i, j = 0…4; совпадает с эталоном.

• A3 — (i + j) mod 5; совпадает с эталоном.

• A4 — (i + j) mod 10 размером 10×10; совпадает с эталоном.

• A2 и A5 — отличаются от эталона (см. замечания ниже).

[56]:
```
function outer(x, y, operation)
    return [operation(i, j) for i in x, j in y]
end

A1 = outer(0:4, 0:4, +)
println("A1:\n", A1)

A2 = outer(0:4, 1:5, (x, y) -> x == 0 ? 0 : x^(y-1))
println("\nA2:\n", A2)

A3 = outer(0:4, 0:4, (x, y) -> (x + y) % 5)
println("\nA3:\n", A3)

A4 = outer(0:9, 0:9, (x, y) -> (x + y) % 10)
println("\nA4:\n", A4)

A5 = outer(1:10, 1:10, (x, y) -> (x - y + 10) % 10)
println("\nA5:\n", A5)
```

```
A1:
[0 1 2 3 4; 1 2 3 4 5; 2 3 4 5 6; 3 4 5 6 7; 4 5 6 7 8]

A2:
[0 0 0 0 0; 1 1 1 1 1; 1 2 4 8 16; 1 3 9 27 81; 1 4 16 64 256]

A3:
[0 1 2 3 4; 1 2 3 4 0; 2 3 4 0 1; 3 4 0 1 2; 4 0 1 2 3]

A4:
[0 1 2 3 4 5 6 7 8 9; 1 2 3 4 5 6 7 8 9 0; 2 3 4 5 6 7 8 9 0 1; 3 4 5 6 7 8
9 0 1 2; 4 5 6 7 8 9 0 1 2 3; 5 6 7 8 9 0 1 2 3 4; 6 7 8 9 0 1 2 3 4 5; 7 8
9 0 1 2 3 4 5 6; 8 9 0 1 2 3 4 5 6 7; 9 0 1 2 3 4 5 6 7 8]

A5:
[0 9 8 7 6 5 4 3 2 1; 1 0 9 8 7 6 5 4 3 2; 2 1 0 9 8 7 6 5 4 3; 3 2 1 0 9 8
7 6 5 4; 4 3 2 1 0 9 8 7 6 5; 5 4 3 2 1 0 9 8 7 6; 6 5 4 3 2 1 0 9 8 7; 7 6
5 4 3 2 1 0 9 8; 8 7 6 5 4 3 2 1 0 9; 9 8 7 6 5 4 3 2 1 0]
```

Рисунок 12 — image-3.png

Замечания к результатам 3.4.8:

• Для A2 эталон требует i^j при j = 1…5 (вторая строка 2 4 8 16 32). В реализации используется x^(y-1), что даёт строку 1 2 4 8 16.

• Для A5 эталон — матрица 9×9 с элементами (i − j) mod 9. В реализации получена матрица 10×10 с mod 10.

• Функция outer построена как поэлементная операция над парами, а не как обобщение матричного умножения, которое требует задание (outer(A, B,) должна быть эквивалентна AB).

3.4.9. Система линейных уравнений

Матрица A[i,j] = |i − j| + 1, вектор y = [7, −1, −3, 5, 17].

```
n_eq = 5
A_sys = [abs(i - j) + 1 for i in 1:n_eq, j in 1:n_eq]
y_sys = [7, -1, -3, 5, 17]
x_sys = A_sys \ y_sys
println(x_sys)
```

Результат: x ≈ [−2, 3, 5, 2, −4] (вычисления в вещественных числах дают погрешности порядка 10^{-15}, например −1.9999999999999987).

```
5 4 3 2 1 0 9 8; 8 7 6 5 4 3 2 1 0 9; 9 8 7 6 5 4 3 2 1 0]
```

```
[57]: n_eq = 5
A_sys = [abs(i - j) + 1 for i in 1:n_eq, j in 1:n_eq]
y_sys = [7, -1, -3, 5, 17]
x_sys = A_sys \ y_sys
println(x_sys)
```

```
[-1.9999999999999987, 2.9999999999999996, 4.999999999999998, 2.000000000000
001, -4.0]
```

```
[58]: M = rand(1:10, 6, 10)
println(M)

N_val = 4
gt_N_counts = [count(>(N_val), M[i, :]) for i in 1:size(M, 1)]
println(gt_N_counts)

M_val = 7
rows_with_two_7 = findall(i -> count(==(M_val), M[i, :]) == 2, 1:size(M, 1)
println(rows_with_two_7)

K_val = 75
col_pairs = []
num_cols = size(M, 2)
for j1 in 1:num_cols
    for j2 in (j1+1):num_cols
        if sum(M[:, j1]) + sum(M[:, j2]) > K_val
            push!(col_pairs, (j1, j2))
        end
    end
end
println(col_pairs)
```

```
[10 10 5 8 10 2 6 5 6 10; 7 7 10 7 7 10 7 5 2 1; 6 9 10 8 5 4 1 8 9 10; 9 6
2 7 5 10 1 8 5 5; 3 1 7 3 4 5 3 6 2 4; 4 1 8 6 10 2 3 9 6 9]
[9, 8, 8, 8, 3, 6]
Int64[]
Any[(1, 3), (1, 4), (1, 5), (1, 8), (1, 10), (2, 3), (3, 4), (3, 5), (3,
8), (3, 10), (4, 5), (4, 8), (4, 10), (5, 8), (5, 10), (8, 10)]
```

Рисунок 13 — image-2.png

3.4.10. Случайная матрица M

```
M = rand(1:10, 6, 10)
N_val = 4
gt_N_counts = [count(>(N_val), M[i, :]) for i in 1:size(M, 1)]

M_val = 7
rows_with_two_7 = findall(i -> count(==(M_val), M[i, :]) == 2, 1:size(M, 1))

K_val = 75
col_pairs = []
num_cols = size(M, 2)
for j1 in 1:num_cols
    for j2 in (j1+1):num_cols
        if sum(M[:, j1]) + sum(M[:, j2]) > K_val
            push!(col_pairs, (j1, j2))
        end
    end
end
```

Результаты для данного запуска (генератор не зафиксирован, поэтому при повторном запуске результаты будут другими):

- число элементов, больших 4, по строкам: [9, 8, 8, 8, 3, 6];
- строки, где число 7 встречается ровно 2 раза: пустой список [];
- пары столбцов с суммой больше 75: (1,3), (1,4), (1,5), (1,8), (1,10), (2,3), (3,4), (3,5), (3,8), (3,10), (4,5), (4,8), (4,10), (5,8), (5,10), (8,10).

Замечание: в условии «сумма пары столбцов» понимается как сумма всех элементов двух столбцов.

```
5 4 3 2 1 0 9 8; 8 7 6 5 4 3 2 1 0 9; 9 8 7 6 5 4 3 2 1 0]
```

[57]:
```
n_eq = 5
A_sys = [abs(i - j) + 1 for i in 1:n_eq, j in 1:n_eq]
y_sys = [7, -1, -3, 5, 17]
x_sys = A_sys \ y_sys
println(x_sys)
```

```
[-1.9999999999999987, 2.9999999999999996, 4.999999999999998, 2.000000000000001, -4.0]
```

[58]:
```
M = rand(1:10, 6, 10)
println(M)

N_val = 4
gt_N_counts = [count(>(N_val), M[i, :]) for i in 1:size(M, 1)]
println(gt_N_counts)

M_val = 7
rows_with_two_7 = findall(i -> count(==(M_val), M[i, :]) == 2, 1:size(M, 1))
println(rows_with_two_7)

K_val = 75
col_pairs = []
num_cols = size(M, 2)
for j1 in 1:num_cols
    for j2 in (j1+1):num_cols
        if sum(M[:, j1]) + sum(M[:, j2]) > K_val
            push!(col_pairs, (j1, j2))
        end
    end
end
println(col_pairs)
```

```
[10 10 5 8 10 2 6 5 6 10; 7 7 10 7 7 10 7 5 2 1; 6 9 10 8 5 4 1 8 9 10; 9 6 2 7 5 10 1 8 5 5; 3 1 7 3 4 5 3 6 2 4; 4 1 8 6 10 2 3 9 6 9]
[9, 8, 8, 8, 3, 6]
Int64[]
Any[(1, 3), (1, 4), (1, 5), (1, 8), (1, 10), (2, 3), (3, 4), (3, 5), (3, 8), (3, 10), (4, 5), (4, 8), (4, 10), (5, 8), (5, 10), (8, 10)]
```

Рисунок 14 — image-2.png

3.4.11. Двойные суммы

```
sum_1 = sum(i^4 / (3 + j) for i in 1:20, j in 1:5)
sum_2 = sum(i^4 / (3 + i * j) for i in 1:20, j in 1:5)
```

Результаты: sum_1 ≈ 639215.2833333338, sum_2 ≈ 89912.02146097131.

```
8), (3, 10), (4, 5), (4, 8), (4, 10), (5, 8), (5, 10), (8, 10)]
```

[59]:
```
sum_1 = sum(i^4 / (3 + j) for i in 1:20, j in 1:5)
println(sum_1)

sum_2 = sum(i^4 / (3 + i * j) for i in 1:20, j in 1:5)
println(sum_2)
```

```
639215.2833333338
89912.02146097131
```

[ ]:

Рисунок 15 — image-1.png

Выводы

В ходе лабораторной работы освоены циклы while и for, условные операторы, тернарный оператор, определение функций, функции высшего порядка map и broadcast, матричные операции (степень, транспонирование, произведение), решение системы линейных уравнений и подключение пакетов Colors и LinearAlgebra.

Примеры раздела 3.2 повторены и проверены. Задания 3.4.2–3.4.7, 3.4.9–3.4.11 выполнены, результаты совпадают с ожидаемыми, за исключением замечаний,

отмеченных в п. 3.4.8. Для задания 3.4.10 результат зависит от случайного запуска, поэтому для воспроизводимости следует зафиксировать генератор (Random.seed!).

Отмечены типичные особенности Julia: A^3 — матричная степень, а не поэлементная; sort! изменяет массив на месте; результаты вещественной арифметики содержат погрешности округления.